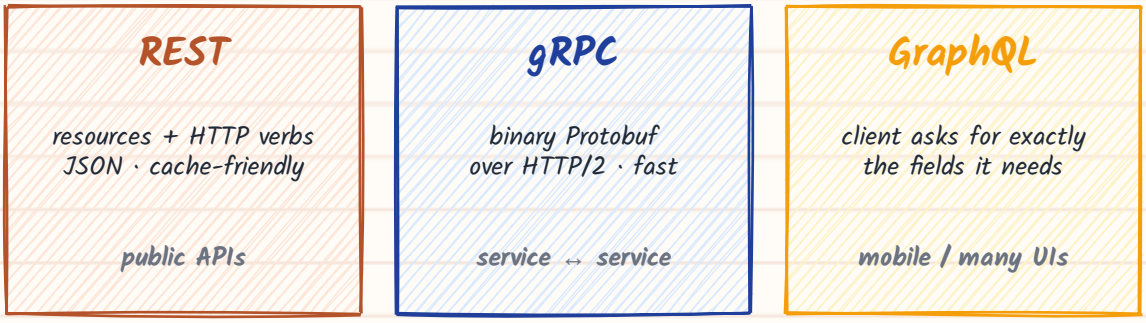
# API Design

Lesson 9 · REST, gRPC, GraphQL, WebSockets & SSE

**In this lesson →** ① The API step ② REST · gRPC · GraphQL ③ REST endpoints ④ Real-time options ⑤ Pagination ⑥ Idempotency ⑦ Versioning & errors ⑧ Protect the API ⑨ Interview Q&A

**Running project:** SnapLink, our photo-sharing app that grows from 1 server to 100M users.

## 1 API styles




| Style | Strengths | Weak spots | Use for |
|---|---|---|---|
| REST | simple, HTTP caching, universal | over/under-fetching | public APIs |
| gRPC | typed, binary, HTTP/2 streaming | not browser-native | service ↔ service |
| GraphQL | client picks fields, one endpoint | caching, rate limits, N+1 | many UIs / mobile |

## 2 SnapLink REST API

```
POST   /v1/photos                  → 201 Created
GET    /v1/users/{id}/feed?cursor= → 200 OK
POST   /v1/users/{id}/followers    → 201
DELETE /v1/users/{id}/followers/me → 204
GET    /v1/photos/{id}             → 200 / 404
```

## 3 Real-time options

**Short polling** — "anything new?" every 5 s
*simple, wasteful*

**Long polling** — server holds request until news
*fewer empty replies*

**SSE** — server → client stream (HTTP)
*live feeds, notifications*

**WebSocket** — two-way, always open
*chat, games, collaboration*

| Option | Direction | Use |
|---|---|---|
| Short polling | client asks repeatedly | simple, low-frequency updates |
| Long polling | server holds request | legacy real-time |
| SSE | server → client | feeds, notifications, live scores |
| WebSocket | both ways | chat, games, collaborative editing |

**Scale note**
Millions of open connections need connection servers + a pub/sub layer to route messages (see SD-27 chat).

## 4 Pagination

**Offset**

?page=500&size=20

DB skips 10,000 rows (slow)
new posts → duplicates

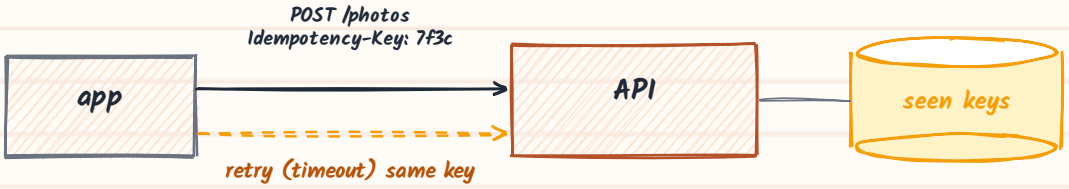
**Cursor**

?after=post_98231&size=20

WHERE id < cursor (fast)
stable while data changes

- ✓ Cursor = opaque token (e.g. last id + timestamp)
- ✓ Query: WHERE id < :cursor ORDER BY id DESC LIMIT 20
- ✓ Offset OK only for small, static lists

## 5 Idempotency




same key → same result, no duplicate post

| Method | Idempotent? |
|---|---|
| GET, PUT, DELETE | yes (by definition) |
| POST | no → use an Idempotency-Key header |
| PATCH | depends on the operation |

## 6 Versioning & errors

→ Version in path (/v1) or header
→ Only additive changes within a version
→ Breaking change → new version; deprecate old slowly
→ One error shape: code, message, details

## 7 Protect the API

✓ Auth: OAuth2 / JWT tokens
✓ Rate limit per user/key → 429 + Retry-After
✓ Validate input sizes (photo ≤ 20 MB)
✓ Do it once in an API gateway

## 8 Interview Corner

**Q. REST vs gRPC vs GraphQL?**

A. REST for public, cacheable APIs; gRPC for fast typed internal calls; GraphQL when clients need flexible field selection.

**Q. WebSocket vs SSE vs long polling?**

A. WebSocket is two-way; SSE is server→client over HTTP; long polling emulates push with held requests.

**Q. Why cursor pagination?**

A. Uses an index (fast at depth) and stays stable when new items arrive.

**Q. How to make POST retries safe?**

A. Client sends an idempotency key; server stores it with the result and returns the same result on retry.

**Q. How do you version an API?**

A. Version in the path or header; additive changes only; new major version for breaking changes.

### ⚡ Revise in 60 seconds

- → **REST**: public
- → **gRPC**: internal, fast
- → **GraphQL**: flexible clients
- → SSE ↓ · WebSocket ↕
- → **Cursor** pagination
- → **Idempotency-Key** on POST
- → **/v1**, additive changes
- → 429 + Retry-After

Next lesson → SD-10
**SQL vs NoSQL**: choosing the right database.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**